Bitácora de Pruebas Técnicas (Proyecto Cafetería: “COFFEE PAW!!”) -

Dilan Alejandro González Alatriste (202521115)

Esta plantilla está diseñada para documentar de manera concisa la ejecución de pruebas técnicas para componentes o módulos específicos.

1. Evidencia de Instanciación

Detalle la creación e inicialización de los objetos de la cafetería: Bebidas, Postres, Clientes y Empleados. ***(10 por cada uno)***

Bebidas:

```
b1 = Bebida(101, "Flat White", 65.0, "Chico", "CALIENTE", ["Leche de avena"])
b2 = Bebida(102, "Dirty Chai Latte", 75.0, "Grande", "CALIENTE", ["Shot extra de espresso"])
b3 = Bebida(103, "Matcha Tonic", 70.0, "Mediano", "FRIA", ["Rodaja de limón", "Hielo"])
b4 = Bebida(104, "Affogato Especial", 85.0, "Chico", "FRIA", ["Helado de vainilla", "Nuez"])
b5 = Bebida(105, "Cold Brew Lavanda", 65.0, "Grande", "FRIA", ["Jarabe de lavanda"])
b6 = Bebida(106, "Frappé de Taro", 80.0, "Grande", "FRIA", ["Crema batida", "Perlas de tapioca"])
b7 = Bebida(107, "Espresso Romano", 45.0, "Chico", "CALIENTE", ["Piel de limón"])
b8 = Bebida(108, "Caramel Macchiato", 70.0, "Grande", "CALIENTE", ["Extra caramelo"])
b9 = Bebida(109, "Limonada de Café", 55.0, "Mediano", "FRIA", ["Menta fresca"])
b10 = Bebida(110, "Chocolate Abuelita", 50.0, "Grande", "CALIENTE", ["Malvaviscos", "Canela"])
```

Empleados:

```
# Estructura: (ID, Nombre, Email, Código, Puesto)
e1 = Empleado(401, "Carmy Berzatto", "carmy@cafeteria.com", "EMP01", "GERENTE")
e2 = Empleado(402, "Sydney Adamu", "sydney@cafeteria.com", "EMP02", "BARISTA")
e3 = Empleado(403, "Ted Lasso", "ted@cafeteria.com", "EMP03", "MESERO")
e4 = Empleado(404, "Claire Dunphy", "claire@cafeteria.com", "EMP04", "GERENTE")
e5 = Empleado(405, "Richie Jerimovich", "richie@cafeteria.com", "EMP05", "MESERO")
e6 = Empleado(406, "Marcus Brooks", "marcus@cafeteria.com", "EMP06", "BARISTA")
e7 = Empleado(407, "Donna Paulsen", "donna@cafeteria.com", "EMP07", "GERENTE")
e8 = Empleado(408, "Nick Miller", "nick@cafeteria.com", "EMP08", "BARISTA")
e9 = Empleado(409, "April Ludgate", "april@cafeteria.com", "EMP09", "MESERO")
e10 = Empleado(410, "Gunther", "gunther@cafeteria.com", "EMP10", "BARISTA")
```

https://github.com/DAGA-Mx/CAFETERIA-COFFEE-PAW

Clientes:

```
c1 = Cliente(301, "Dilan Gonzalez", "dilan@gmail.com", 150)
c2 = Cliente(302, "Edgar Martinez", "carlos@email.com", 50)
c3 = Cliente(303, "Ana Sanchez", "ana@email.com", 300)
c4 = Cliente(304, "Willem Kmetsch", "willem@leprechaun.com", 500)
c5 = Cliente(305, "Jimena Rodriguez", "Jimez0@gmail.com", 20)
c6 = Cliente(306, "Scott Pilgrim", "pilgrim@sxbmb.com", 100)
c7 = Cliente(307, "Gerardo Lopez", "Gerry@gmail.com", 80)
c8 = Cliente(308, "Carlos Santana", "carl00sx@datacracks.com", 250)
c9 = Cliente(309, "Minerva Díaz", "minne@centlop.com", 180)
c10 = Cliente(310, "Pedro Perez", "peperez@email.com", 10)
```

Postres:

```
p1 = Postre(201, "Avocado Toast con Huevo", 95.0, False, False)
p2 = Postre(202, "Galleta Choco-Sea Salt", 35.0, False, False)
p3 = Postre(203, "Cruasán de Almendras", 55.0, False, False)
p4 = Postre(204, "Brownie de Matchay Chocolate", 65.0, True, False)
p5 = Postre(205, "Muffin Keto de Berries", 60.0, False, True)
p6 = Postre(206, "Panqué de Plátano y Nuez", 45.0, True, False)
p7 = Postre(207, "Tarta de Frutos Rojos", 75.0, False, True)
p8 = Postre(208, "Babka de Chocolate", 55.0, False, False)
p9 = Postre(209, "Quiche de Tomate Deshidratado", 85.0, False, False)
p10 = Postre(210, "New York Cheesecake con Maracuyá", 80.0, False, False)
```

IMPRESIÓN DE CADA UNO:

```
>>> CARGANDO MENÚ DE BEBIDAS...
ID: 101 | Flat White ($65.0) | Cat: Bebida CALIENTE
ID: 102 | Dirty Chai Latte ($75.0) | Cat: Bebida CALIENTE
ID: 103 | Matcha Tonic ($70.0) | Cat: Bebida FRIA
ID: 104 | Affogato Especial ($85.0) | Cat: Bebida FRIA
ID: 105 | Cold Brew Lavanda ($65.0) | Cat: Bebida FRIA
ID: 106 | Frappé de Taro ($80.0) | Cat: Bebida FRIA
ID: 107 | Espresso Romano ($45.0) | Cat: Bebida CALIENTE
ID: 108 | Caramel Macchiato ($70.0) | Cat: Bebida CALIENTE
ID: 109 | Limonada de Café ($55.0) | Cat: Bebida FRIA
ID: 110 | Chocolate Abuelita ($50.0) | Cat: Bebida CALIENTE

>>> CARGANDO MENÚ DE POSTRES (10 OBJETOS)
ID: 201 | Avocado Toast con Huevo ($95.0) | Cat: Postre
ID: 202 | Galleta Choco-Sea Salt ($35.0) | Cat: Postre
ID: 203 | Cruasán de Almendras ($55.0) | Cat: Postre
ID: 204 | Brownie de Matchay Chocolate ($65.0) | Cat: Postre
ID: 205 | Muffin Keto de Berries ($60.0) | Cat: Postre
ID: 206 | Panqué de Plátano y Nuez ($45.0) | Cat: Postre
ID: 207 | Tarta de Frutos Rojos ($75.0) | Cat: Postre
ID: 208 | Babka de Chocolate ($55.0) | Cat: Postre
ID: 209 | Quiche de Tomate Deshidratado ($85.0) | Cat: Postre
ID: 210 | New York Cheesecake con Maracuyá ($80.0) | Cat: Postre
```

https://github.com/DAGA-Mx/CAFETERIA-COFFEE-PAW

```
>>> ALTA CLIENTES...
ID: 301 | Dilan Gonzalez | Puntos: 150
ID: 302 | Edgar Martinez | Puntos: 50
ID: 303 | Ana Sanchez | Puntos: 300
ID: 304 | Willem Kmetsch | Puntos: 500
ID: 305 | Jimena Rodriguez | Puntos: 20
ID: 306 | Scott Pilgrim | Puntos: 100
ID: 307 | Gerardo Lopez | Puntos: 80
ID: 308 | Carlos Santana | Puntos: 250
ID: 309 | Minerva Díaz | Puntos: 180
ID: 310 | Pedro Perez | Puntos: 10

>>> DANDO DE ALTA EMPLEADOS...
ID: EMP01 | Carmy Berzatto | Rol: GERENTE
ID: EMP02 | Sydney Adamu | Rol: BARISTA
ID: EMP03 | Ted Lasso | Rol: MESERO
ID: EMP04 | Claire Dunphy | Rol: GERENTE
ID: EMP05 | Richie Jerimovich | Rol: MESERO
ID: EMP06 | Marcus Brooks | Rol: BARISTA
ID: EMP07 | Donna Paulsen | Rol: GERENTE
ID: EMP08 | Nick Miller | Rol: BARISTA
ID: EMP09 | April Ludgate | Rol: MESERO
ID: EMP10 | Gunther | Rol: BARISTA

--- VALIDACIÓN DE DATOS FINALIZADA ---
```

https://github.com/DAGA-Mx/CAFETERIA-COFFEE-PAW

2. Pruebas de Métodos Individuales

Registro de la ejecución de métodos individuales dentro del objeto/módulo.

Clase de cliente: Capaz de realizar pedidos, consultar historial, canjear puntos, etc.

```python
class Cliente(Persona):
    def __init__(self, idPersona, nombre, email, puntosFidelidad):
        super().__init__(idPersona, nombre, email)
        self.puntosFidelidad = puntosFidelidad
        self.historialPedidos = []

    def realizarPedido(self, pedido):
        self.historialPedidos.append(pedido)
        return f"El cliente {self.nombre} ha realizado un nuevo pedido."

    def consultarHistorial(self):
        return f"Consultando el historial de {self.nombre}... Total de pedidos: {len(self.historialPedidos)}"

    def canjearPuntos(self, puntos_a_usar):
        if self.puntosFidelidad >= puntos_a_usar:
            self.puntosFidelidad -= puntos_a_usar
            return f"Se han canjeado {puntos_a_usar} puntos. Puntos restantes: {self.puntosFidelidad}"
        return "Puntos insuficientes para el canje."

    def mostrar_detalle(self):
        return f"ID: {self.idPersona} | {self.nombre} | Puntos: {self.puntosFidelidad}"
```

Prueba de métodos de cliente: Realizada con éxito, sin complicaciones en código.

```
>>> INICIANDO PROCESO DE PEDIDO...
Empleado en caja: Sydney Adamu (BARISTA)
Cliente: Dilan Gonzalez (Puntos: 150)

[SISTEMA]: Seleccionando productos...
 - Agregado: Flat White
 - Agregado: Frappé de Taro
 - Agregado: Cruasán de Almendras
Verificando ingedrientes disponibles para el pedido #995... ¡Stock confirmado!

>>> APLICANDO GESTIÓN COMERCIAL...
El total del pedido #995 ha sido recalculado: $215.00
¿Desea canjear puntos a crédito?: SI (Canjeando 50 puntos = $50 de descuento)
[SISTEMA]: ¡ÉXITO! - Descuento aplicado.
```

https://github.com/DAGA-Mx/CAFETERIA-COFFEE-PAW

Clase de Empleado: Capaz de actualizar inventario, cambiar estado del pedido, etc.

```python
class Empleado(Persona):
    def __init__(self, idPersona, nombre, email, idEmpleado, rol):
        super().__init__(idPersona, nombre, email)
        self.idEmpleado = idEmpleado
        self.rol = rol

    def actualizarInventario(self):
        if self.rol == "GERENTE":
            return f"[{self.rol}] {self.nombre} está actualizando el inventario maestro."
        return f"Acceso denegado: {self.nombre} no tiene permisos de GERENTE."

    def cambiarEstadoPedido(self, nuevo_estado):
        return f"El empleado {self.nombre} cambió el estado del pedido a: {nuevo_estado}"

    def mostrar_detalle(self):
        return f"ID: {self.idEmpleado} | {self.nombre} | Rol: {self.rol}"
```

Prueba de métodos de Empleado: Realizada con éxito.

```
[COCINA]: El empleado Marcus Brooks cambió el estado del pedido a: PREPARANDO
[COCINA]: El empleado Marcus Brooks cambió el estado del pedido a: ENTREGADO
```

3. Prueba de Retos (Escenarios Complejos)

Documentación de pruebas que involucran interacciones complejas o flujos de trabajo completos.
Se añade la primera función: agregar extra
Junto a “calcularPrecioFinal” y mostrar detalle.

```python
### 07 DE MARZO
    def agregarExtra(self, extra):
        self.modificadores.append(extra)
        return f"Extra '{extra}' agregado a la bebida {self.nombre}."

    def calcularPrecioFinal(self):

        costo_extras = len(self.modificadores) * 5.0
        precio_final = self.precioBase + costo_extras
        return precio_final

    def mostrar_detalle(self):
        return f"ID: {self.idProducto} | {self.nombre} ($${self.precioBase}) | Cat: Bebida {self.temperatura}"

class Postre(ProductoBase):
    def __init__(self, idProducto, nombre, precioBase, esVegano, sinGluten):
        super().__init__(idProducto, nombre, precioBase)
        self.esVegano = esVegano
        self.sinGluten = sinGluten

    def mostrar_detalle(self):
        return f"ID: {self.idProducto} | {self.nombre} ($${self.precioBase}) | Cat: Postre"
```

https://github.com/DAGA-Mx/CAFETERIA-COFFEE-PAW

Se concretaron los métodos de lógica para la clase “Pedido”:
Se calcularía el total de los productos seleccionados, sumándolos, y mostrando un recálculo.

```python
class Pedido:
    def __init__(self, idPedido, productos, estado, total):
        self.idPedido = idPedido
        self.productos = productos
        self.estado = estado
        self.total = total

    def calcularTotal(self):
        suma = 0
        for producto in self.productos:

            if isinstance(producto, Bebida):
                suma += producto.calcularPrecioFinal()
            else:
                suma += producto.precioBase
        self.total = suma
        return f"El total del pedido #{self.idPedido} ha sido recalculado: ${self.total:.2f}"

    def validarStock(self, inventario_actual):

        return f"Verificando ingedrientes disponibles para el pedido #{self.idPedido}... ¡Stock confirmado!"
```

Se trabajó mucho en pulir las acciones de al elegir varios productos: Se generaban bucles del pedido:

```
Artículos: ['Flat White', 'Frappé de Taro', 'Cruasán de Almendras']
Total Final: $165.00
Estado: PAGADO. Ticket generado en PDF.
IMPRIMIENDO TICKET...
Artículos: ['Flat White', 'Frappé de Taro', 'Cruasán de Almendras']
Total Final: $165.00
Estado: PAGADO. Ticket generado en PDF.
Artículos: ['Flat White', 'Frappé de Taro', 'Cruasán de Almendras']
Total Final: $165.00
Artículos: ['Flat White', 'Frappé de Taro', 'Cruasán de Almendras']
Total Final: $165.00
Artículos: ['Flat White', 'Frappé de Taro', 'Cruasán de Almendras']
Total Final: $165.00
Artículos: ['Flat White', 'Frappé de Taro', 'Cruasán de Almendras']
Total Final: $165.00
Artículos: ['Flat White', 'Frappé de Taro', 'Cruasán de Almendras']
Total Final: $165.00
Artículos: ['Flat White', 'Frappé de Taro', 'Cruasán de Almendras']
Artículos: ['Flat White', 'Frappé de Taro', 'Cruasán de Almendras']
Artículos: ['Flat White', 'Frappé de Taro', 'Cruasán de Almendras']
Total Final: $165.00
Estado: PAGADO. Ticket generado en PDF.
IMPRIMIENDO TICKET...
PS C:\Users\dilon>
```

https://github.com/DAGA-Mx/CAFETERIA-COFFEE-PAW

Finalmente se terminó y mostró los resultados esperados:

```
Artículos: ['Flat White', 'Frappé de Taro', 'Cruasán de Almendras']
Total Final: $165.00
Estado: PAGADO. Ticket generado en PDF.
IMPRIMIENDO TICKET...
PS C:\Users\dilon>
```

Detalle de seguimiento del programa:

Se subió el código en un repositorio de github:

https://github.com/DAGA-Mx/CAFETERIA-COFFEE-PAW

https://github.com/DAGA-Mx/CAFETERIA-COFFEE-PAW